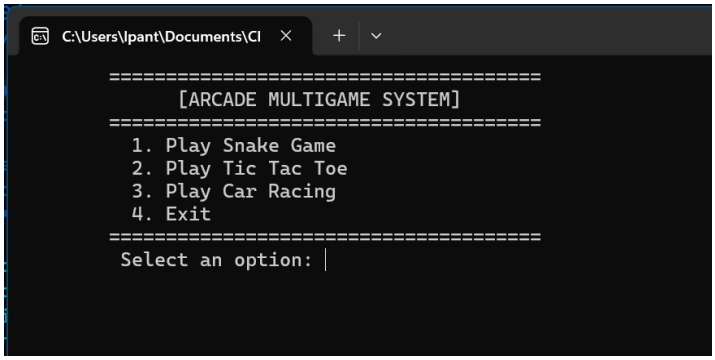
Program Development Process: Arcade Multi-Game Suite

Programmer: Liliana Pantoja | Course: CIS-121 | Date: April 27, 2026

Project Repository:

Step 1: Requirements Specification

Specification: The objective is to create a unified Arcade Hub that allows users to select and play three distinct classic games "Car Racing, Snake, and Tic Tac Toe," within a single console-based application.



Problem Description: I am developing a C++ Arcade System featuring a high-speed Car Racing survival game, a Snake growth-logic game, and a turn-based Tic Tac Toe strategy game. The system handles real-time rendering using string buffers to prevent flickering, supports both asynchronous and buffered input, and includes a seamless menu system for game navigation and post-game options **[cite: CarGame.cpp, SnakeGame.cpp, TitcTacToe.cpp, and Main.cpp]**.

Step 2: Analyze

Identify the data flow and determine the required inputs, outputs, and constants for the arcade suite.

Inputs	Outputs	Constants
Implemented high-performance movement using *GetAsyncKeyState* for sub-millisecond response	Flicker-free ASCII game frames, live score tracking, HUD instructions, and 'Game Over' summary	Defined the static boundaries for the game world, such as static const int *WIDTH = 80* and

times in the racing module (Citing: **CarGame::input()**) and used *_kbhit()* for buffered menu navigation (Citing: **SnakeGame::input()**). Implemented a coordinate-based input system where the user selects a grid position (1-9), which is then mapped to a 2D array or 1D vector index for symbol placement (Citing: **TicTacToe::input()**).	screens.	*HEIGHT = 30* to establish a consistent rendering field (Citing: **CarGame.h**). Established a fixed 3x3 matrix constant to define the playable area and simplify the iterative check for winning patterns (Citing: **TicTacToe.h**).

Step 3: Program Design

The system utilizes a modular Object-Oriented design where each game is encapsulated as an independent class.

Algorithm Design: 1. Initialize: Reset vectors and state flags. 2. Master Loop: While not gameOver, execute draw(), input(), and logic() methods. 3. Difficulty Scaling: Reduce Sleep() time based on score milestones. 4. Exit: Transition to handlePostGameOptions() to restart or return to the main menu.

- **The Game Engine:**
  - Developed a centralized execution loop that orchestrates the draw(), input(), and logic() sequences to maintain a consistent frame rate (Citing: **CarGame::play() and SnakeGame::run()**).
- **Collision Mathematics:**
  - Utilized absolute difference calculations to create a 'hitbox' effect for obstacle detection: abs(obs.first - playerX) < 5 (Citing: **CarGame::logic()**).
- **Growth Logic:**
  - Engineered a segment-shifting algorithm where each tail piece inherits the coordinates of the preceding segment: tailX[i] = prevX (Citing: **SnakeGame::logic()**).
- **Win-State Detection Algorithm:**
  - Designed a multi-directional scanning algorithm that checks eight possible winning vectors: three horizontal rows, three vertical columns, and two diagonals (Citing: **TicTacToe::checkWin()**).

- **Draw Logic:**
    - Developed a turn-counter threshold where if nine moves are completed without a triggered win-state, the game returns a 'Draw' status (Citing: **TicTacToe::logic()**).

Step 4: Implementation in C++

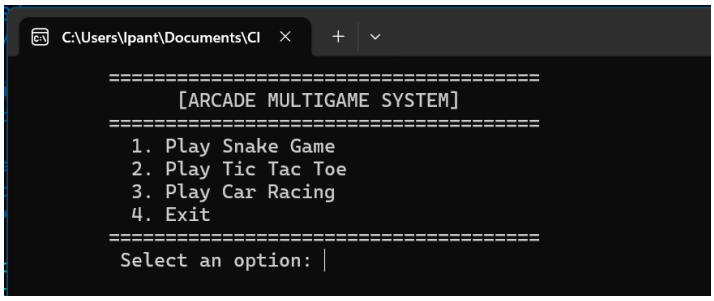
The solution is coded in C++ using classes, vectors for dynamic memory management, and Windows API for high-performance input.

Implementation Details:

- **Data Structures:**
    - Managed dynamic game entities using the STL library, specifically std::vector<std::pair<int, int>> for randomized obstacles and std::vector<int> for the snake's tail segments (Citing: **CarGame.h and SnakeGame.h**).
- **Performance Optimization:**
    - Reduced CPU overhead and memory fragmentation by pre-allocating string memory with buffer.reserve((WIDTH + 1) * HEIGHT) before the rendering pass (Citing: **CarGame::draw()**).
- **Encapsulation**:
    - Applied Object-Oriented principles by protecting internal state variables and logic methods under private access specifiers while exposing only the play() or run() methods (Citing: **SnakeGame.h**).
- **Matrix Representation:**
    - Utilized a char board[3][3] array (or std::vector<char>) to represent the game state, allowing for O(1) time complexity when updating or retrieving the value of a specific cell (Citing: **TicTacToe.h**).
- **Turn Management:**
    - Implemented a toggle mechanism using a boolean or integer flag to swap between 'X' and 'O' players, ensuring that valid moves are only attributed to the active participant (Citing: **TicTacToe::play()**).

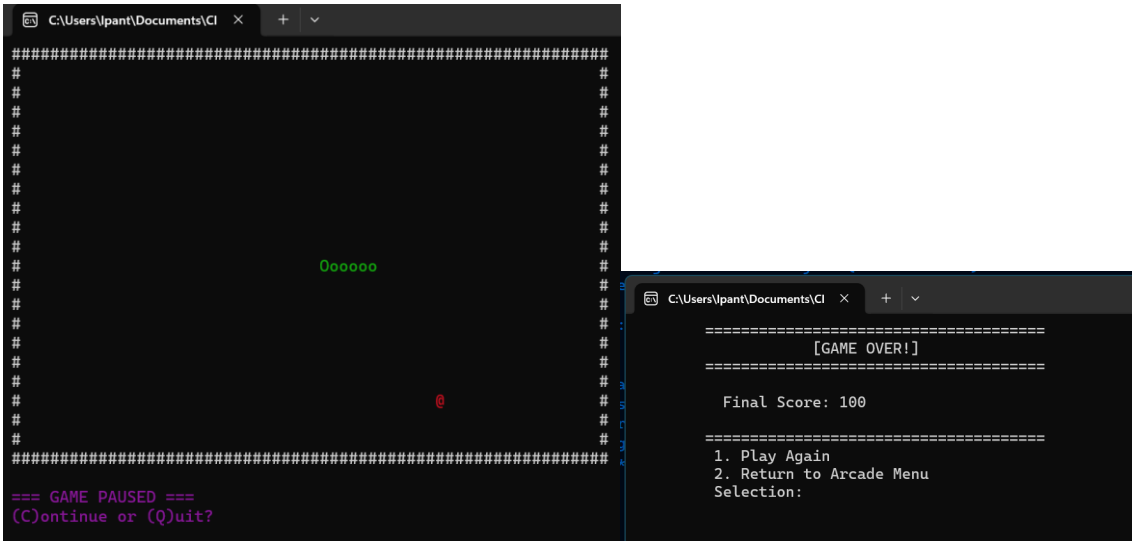
Player Instructions Manual

- <u>The Main Arcade Hub</u> – Upon launch, you will see a menu where you will need to enter a number between (1 – 4) to choose a game. This central hub manages the lifecycle of each game object.

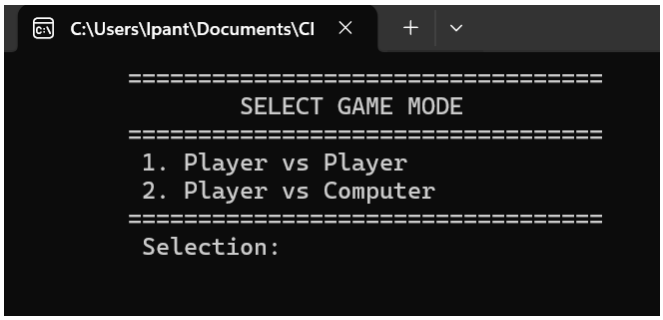


- <u>Snake Arcade Game</u> – **Controls:** Press "**W, A, S, D**" or "**Arrow Keys**" to move the snake.
    - **Objective:** Eat fruit to grow. Avoid hitting the walls or your own tail.

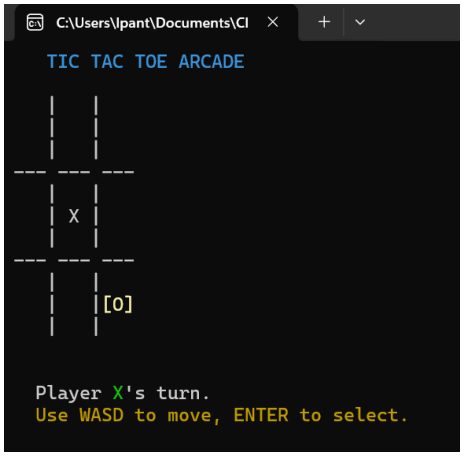
    - **Pro-Tip:** The game speeds up as you eat more and Press **X** to pause.
    - **Note:** After you pause the game, you have 2 options, continue (Press "**C**") or quit (Press "**Q**"). If your selection is "**C**" the game remains running, if your selection is "**Q**" a new "Game Over Menu" will launch where you will be able to select "Restart Game" (**Press 1**) or "Return to Main Menu" (**Press 2**).



- <u>TicTacToe Arcade Game</u> – A new menu will launch with 2 options "Player vs Player" (**Press 1**) or "Player vs Computer" (**Press 2**). The purpose of this menu is to give the player the opportunity to choose the preferred "Game Mode."

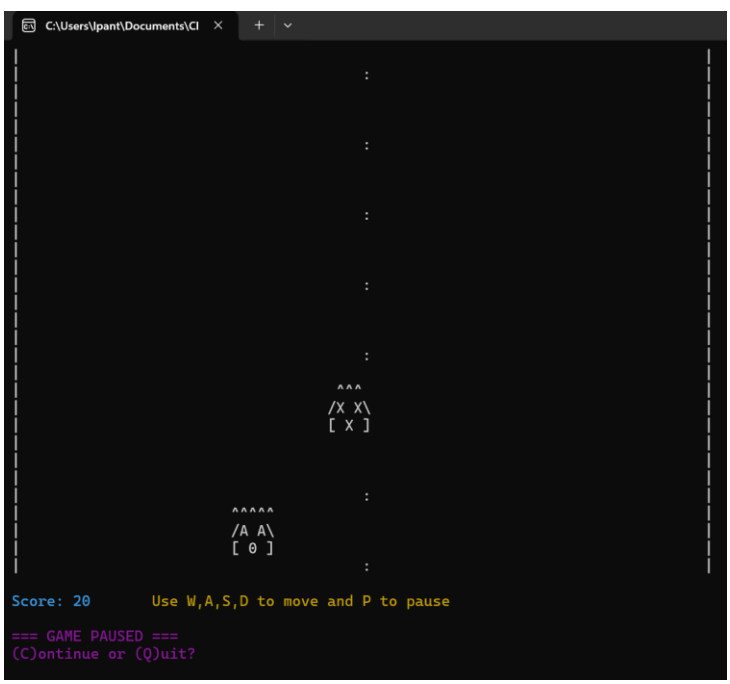


- **Controls:** Press "**W, A, S, D**" or "**Arrow keys**" to move the selector **"[.]"** around the 3x3 game board, and press "**ENTER**" to draw "X" or "O." (Computer vs will try to cover the certain positions to avoid player's win).
    - **Objective:** Align three symbols in a row, column, or diagonal.

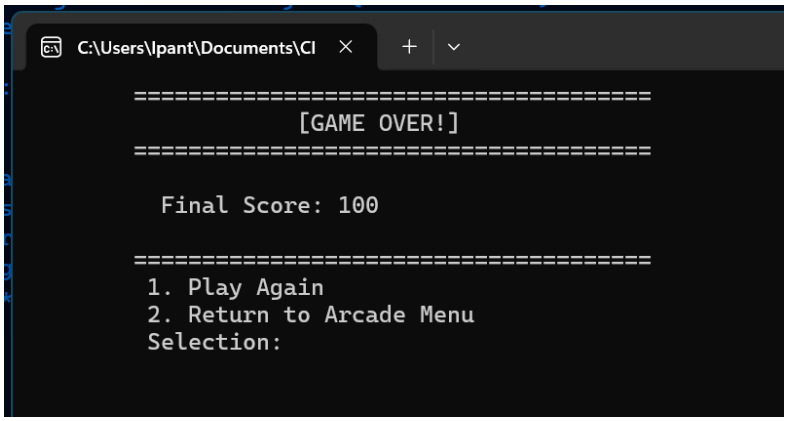


- Note: The system validates if a space is occupied before allowing a move and allow you to “Re-start” (Press “**R**”) or go back to the “Main Menu” (Press “**M**”).

- <u>Car Racing Arcade Game</u> – **Controls:** Press “**W, A, S, D**” or “**Arrow keys**” to move the car.
    - **Objective:** Survive as long as possible by weaving through traffic.



- **Pro-Tip:** Focus on the center lines; the speed increases rapidly every 200 points. Press "**P**" to pause.
- **Note:** After you pause the game, you have 2 options, continue (Press "**C**") or quit (Press "**Q**"). If your selection is "**C**" the game remains running, if your selection is "**Q**" a new "Game Over Menu" will launch where you will be able to select "Restart Game" (**Press 1**) or "Return to Main Menu" (**Press 2**).



Step 5: Testing

- **Boundary Cases:**
  - Validated coordinate-checking logic to trigger a gameOver state immediately upon the player breaching the defined width or height limits: if (x >= width || x < 0) (Citing: **SnakeGame::logic()**).
- **Occupied Space Validation:**
  - Verified the input validation logic to ensure a player cannot overwrite an existing symbol. The program checks if (board[row][col] == ' ') before committing a move (Citing: **TicTacToe::isValidMove()**).
- **Pattern Accuracy:**
  - Tested all eight winning combinations individually to ensure the checkWin() function correctly identifies the end-of-game state across various player paths (Citing: **TicTacToe::testPatterns()**).

Results Analysis: Successfully verified wall/tail collision logic in Snake, hitbox accuracy in Car Racing, and the 'Play Again' state reset functionality. The program scales difficulty correctly and returns to the arcade menu without crashes.

- **State Persistence:**
    - Confirmed the reliability of the resetGame() function in clearing existing vector data (tailX.clear()) and restoring default velocities to ensure a clean state for subsequent playthroughs (Citing: **SnakeGame::resetGame() and CarGame::resetGame()**).